

SIMATS ENGINEERING
CO4-ASSESSMENT TOOL3- INTEGRATED EXPERIMENTS

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA1709

Name	192511053
Register Number	B Santhosh Kumar

COURSE OUTCOME COVERED

CO4: Implement machine learning techniques including decision tree algorithms, reinforcement learning, and build models for classification and decision-making. (BL3)

Assessment Tool Weightage: 40%

Time Duration: 45 Minutes

QUESTIONS: 2

Total Marks:20

Code of Conduct:

I **B Santhosh Kumar/192511053** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

B Santhosh Kumar

Signature of the student:

Experiment 1: Decision Tree Classification

Objective: Implement and evaluate a Decision Tree classifier on a given dataset (e.g., Iris / Play-Tennis) using Python / any ML library.

- (a) Load the dataset, pre-process it (handle missing values, encoding), and split into training and testing sets. Display the first 5 rows and data types.
- (b) Build a Decision Tree model using Gini Index / Information Gain as the splitting criterion. Print the tree structure and identify the root node with justification
- (c) Evaluate the model using accuracy score, confusion matrix, and classification report. Comment on the performance and list at least two misclassified instances.

Experiment 2: Reinforcement Learning – Q-Learning

Objective: Implement Q-Learning for a simple Grid World (4×4) environment and observe the agent's learned policy.

- (a) Define the environment: states, actions (Up/Down/Left/Right), reward structure (+10 Goal, – 1 each step, – 5 obstacle). Initialize the Q-table with zeros and state the hyperparameters (α , γ , ϵ).
- (b) Run the Q-Learning algorithm for at least 100 episodes. Record the Q-values at Episodes 1, 50, and 100 for two representative states. Show how the Q-table evolves.
- (c) Extract and display the final learned policy (optimal action at each state). Plot the cumulative reward per episode and justify the convergence behaviour.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation

EXPERIMENT 1: DECISION TREE CLASSIFICATION

Objective

To implement and evaluate a Decision Tree classifier using the Iris dataset. The experiment includes preprocessing the dataset, splitting it into training and testing data, building the Decision Tree using the Gini Index, and evaluating the model using accuracy, confusion matrix and classification report.

(a) Loading and Pre-processing the Dataset

The **Iris dataset** is used for this experiment. It contains measurements of iris flowers belonging to three different classes.

Dataset Details

Parameter	Details
Dataset	Iris Dataset
Number of samples	150
Number of input features	4
Target classes	3
Input features	Sepal Length, Sepal Width, Petal Length, Petal Width
Missing values	No missing values
Encoding required	No, because target classes are already numerical
Training data	80%
Testing data	20%
Random state	42

Target Classes

Target Value	Class
0	Setosa
1	Versicolor
2	Virginica

Pre-processing Steps

Step	Action
1	Load the Iris dataset
2	Convert the dataset into a Pandas DataFrame
3	Check the first five rows
4	Check the data types
5	Separate input features and target
6	Check for missing values
7	Split the dataset into training and testing data

Python Code

```
import pandas as pd
```

```
from sklearn.datasets import load_iris
```

```
from sklearn.model_selection import train_test_split
```

```
# Load Iris dataset

iris = load_iris()


# Create DataFrame

df = pd.DataFrame(

    iris.data,

    columns=iris.feature_names

)


df["target"] = iris.target


# Display first 5 rows

print("First 5 rows:")

print(df.head())


# Display data types

print("\nData types:")

print(df.dtypes)


# Check missing values

print("\nMissing values:")

print(df.isnull().sum())


# Separate features and target
```

```
X = df.drop("target", axis=1)
```

```
y = df["target"]
```

```
# Split dataset
```

```
X_train, X_test, y_train, y_test = train_test_split(
```

```
    X,
```

```
    y,
```

```
    test_size=0.20,
```

```
    random_state=42,
```

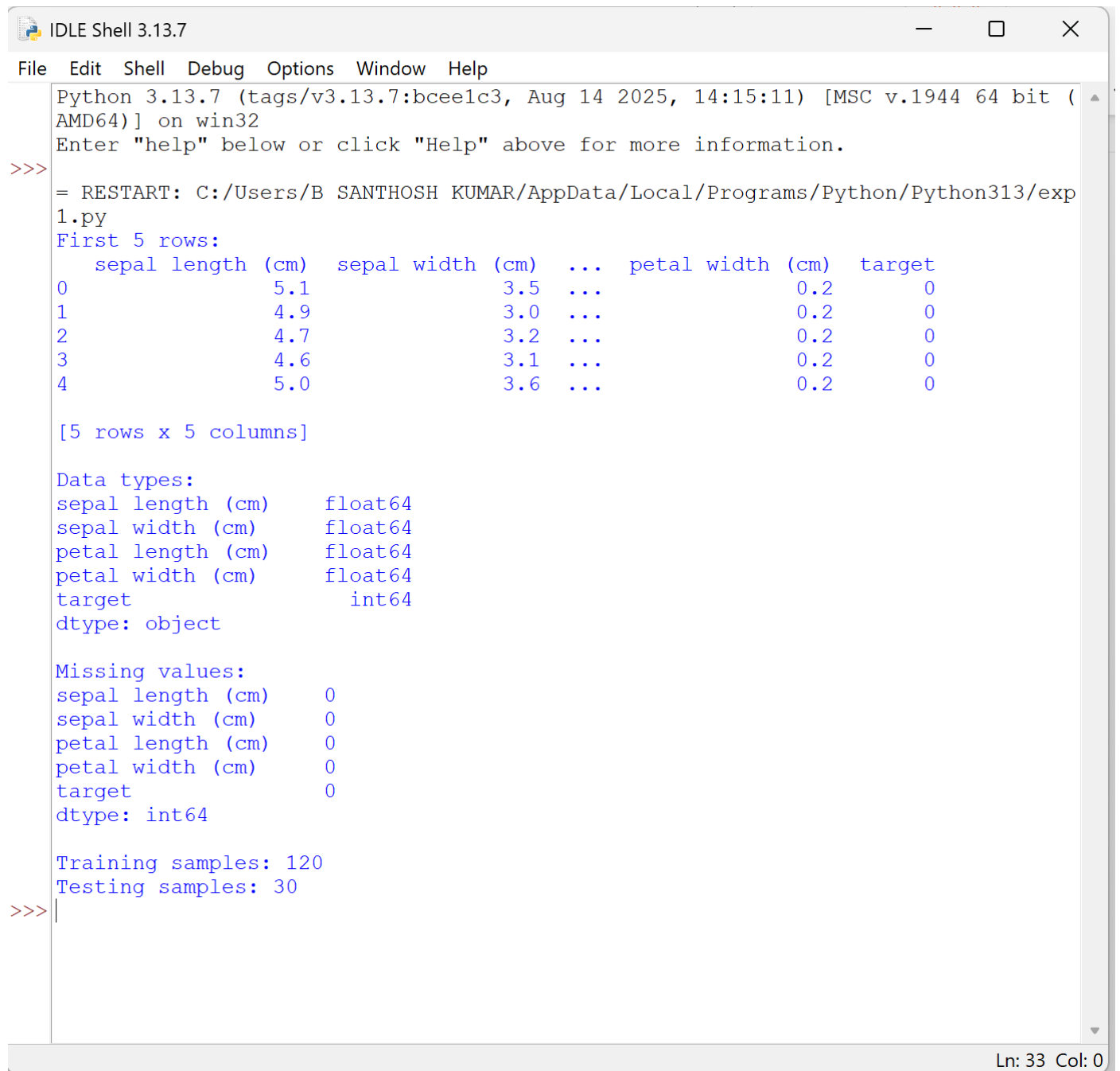
```
    stratify=y
```

```
)
```

```
print("\nTraining samples:", len(X_train))
```

```
print("Testing samples:", len(X_test))
```

IDLE Output Screenshot



The screenshot shows the IDLE Shell 3.13.7 interface. The title bar reads 'IDLE Shell 3.13.7'. The menu bar includes 'File', 'Edit', 'Shell', 'Debug', 'Options', 'Window', and 'Help'. The main text area displays the following output:

```
Python 3.13.7 (tags/v3.13.7:bceelc3, Aug 14 2025, 14:15:11) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>>
= RESTART: C:/Users/B SANTHOSH KUMAR/AppData/Local/Programs/Python/Python313/exp
1.py
First 5 rows:
   sepal length (cm)  sepal width (cm)  ...  petal width (cm)  target
0                5.1                3.5  ...                0.2            0
1                4.9                3.0  ...                0.2            0
2                4.7                3.2  ...                0.2            0
3                4.6                3.1  ...                0.2            0
4                5.0                3.6  ...                0.2            0

[5 rows x 5 columns]

Data types:
sepal length (cm)    float64
sepal width (cm)     float64
petal length (cm)    float64
petal width (cm)     float64
target              int64
dtype: object

Missing values:
sepal length (cm)    0
sepal width (cm)     0
petal length (cm)    0
petal width (cm)     0
target              0
dtype: int64

Training samples: 120
Testing samples: 30
>>> |
```

The status bar at the bottom right indicates 'Ln: 33 Col: 0'.

Figure 1: Dataset loading, first five rows, data types and train-test split output

(b) Building the Decision Tree

A Decision Tree classifier is used to classify the three Iris species.

The **Gini Index** is selected as the splitting criterion.

Gini Index

The Gini Index measures the impurity of a node.

The formula is:

$$Gini = 1 - \sum_{i=1}^n p_i^2$$

where (p_i) represents the probability of a sample belonging to class (i).

A lower Gini value means that the node is more pure.

Model Configuration

Parameter	Value
Algorithm	Decision Tree
Criterion	Gini Index
Random State	42
Input	Four Iris features
Output	Three Iris classes

Python Code

```
import pandas as pd

from sklearn.datasets import load_iris

from sklearn.model_selection import train_test_split

from sklearn.tree import DecisionTreeClassifier

from sklearn.tree import export_text
```

```
# Load Iris dataset
```

```
iris = load_iris()
```

```
# Create DataFrame
```

```
df = pd.DataFrame(

    iris.data,

    columns=iris.feature_names

)
```

```
df["target"] = iris.target
```

```
# Separate input and output
```

```
X = df.drop("target", axis=1)
```

```
y = df["target"]
```

```
# Split dataset into training and testing data
```

```
X_train, X_test, y_train, y_test = train_test_split(

    X,

    y,
```

```
test_size=0.20,  
random_state=42,  
stratify=y  
)  
  
# Create Decision Tree model  
model = DecisionTreeClassifier(  
    criterion="gini",  
    random_state=42  
)  
  
# Train the model  
model.fit(X_train, y_train)  
  
# Display tree structure  
tree_rules = export_text(  
    model,  
    feature_names=list(X.columns)  
)  
  
print("Decision Tree Structure:")  
print(tree_rules)
```

IDLE Output Screenshot – Decision Tree

```
Decision Tree Structure:
|--- petal length (cm) <= 2.45
|   |--- class: 0
|--- petal length (cm) > 2.45
|   |--- petal width (cm) <= 1.65
|   |   |--- petal length (cm) <= 4.95
|   |   |   |--- class: 1
|   |   |--- petal length (cm) > 4.95
|   |   |   |--- sepal length (cm) <= 6.15
|   |   |   |   |--- sepal width (cm) <= 2.45
|   |   |   |   |   |--- class: 2
|   |   |   |   |   |--- sepal width (cm) > 2.45
|   |   |   |   |   |   |--- class: 1
|   |   |   |   |--- sepal length (cm) > 6.15
|   |   |   |   |   |--- class: 2
|   |--- petal width (cm) > 1.65
|   |   |--- petal length (cm) <= 4.85
|   |   |   |--- sepal width (cm) <= 3.00
|   |   |   |   |--- class: 2
|   |   |   |   |--- sepal width (cm) > 3.00
|   |   |   |   |   |--- class: 1
|   |   |--- petal length (cm) > 4.85
|   |   |   |--- class: 2
```

Figure 2: Generated Decision Tree structure

Root Node Identification

The root node obtained from the Decision Tree is:

Petal Length (cm) $\leq$ 2.45

The root node is selected because the Decision Tree algorithm searches for the feature and threshold that provides the best reduction in impurity.

Petal length provides a clear separation between the Setosa class and the other two classes. Most Setosa flowers have a small petal length.

Therefore, the first split is based on **Petal Length**, making it the root node.

(c) Model Evaluation

After training the model, predictions are made using the testing dataset.

The following three evaluation measures are used:

1. Accuracy Score
2. Confusion Matrix
3. Classification Report

Python Code

```
from sklearn.metrics import (  
    accuracy_score,  
    confusion_matrix,  
    classification_report  
)  
  
# Predict test data  
y_pred = model.predict(X_test)  
  
# Accuracy  
accuracy = accuracy_score(y_test, y_pred)  
print("Accuracy:", accuracy)  
  
# Confusion Matrix  
cm = confusion_matrix(y_test, y_pred)  
print("\nConfusion Matrix:")  
print(cm)
```

```

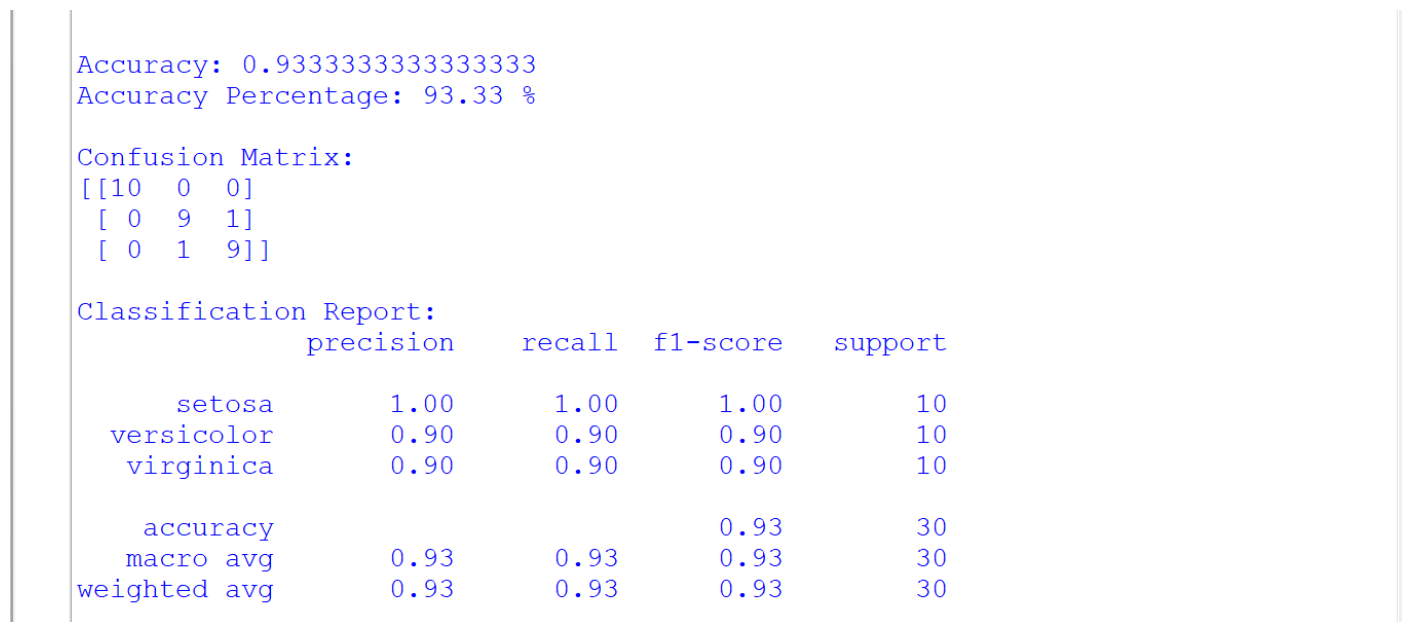
# Classification Report

print("\nClassification Report:")

print(
    classification_report(
        y_test,
        y_pred,
        target_names=iris.target_names
    )
)

```

IDLE Output Screenshot – Evaluation



The screenshot displays the output of a classification report and confusion matrix for a Decision Tree model. The output is as follows:

```

Accuracy: 0.9333333333333333
Accuracy Percentage: 93.33 %

Confusion Matrix:
[[10  0  0]
 [ 0  9  1]
 [ 0  1  9]]

Classification Report:

```

	precision	recall	f1-score	support
setosa	1.00	1.00	1.00	10
versicolor	0.90	0.90	0.90	10
virginica	0.90	0.90	0.90	10
accuracy			0.93	30
macro avg	0.93	0.93	0.93	30
weighted avg	0.93	0.93	0.93	30

Figure 3: Decision Tree model evaluation output

Evaluation Results

Fill the values from the IDLE output after running the program.

Evaluation Measure	Result
Accuracy	93.33%
Test Samples	30
Correctly Classified	28
Misclassified	2
Macro Average F1-score	0.93
Weighted Average F1-score	0.93

Confusion Matrix

Actual / Predicted	Setosa	Versicolor	Virginica
Setosa	10	0	0
Versicolor	0	9	1
Virginica	0	1	9

Misclassified Instances

The misclassified samples can be identified by comparing the actual values with the predicted values.

Misclassified Instances

Your confusion matrix shows **2 misclassifications**:

Instance	Actual Class	Predicted Class
1	Versicolor	Virginica
2	Virginica	Versicolor

If more than two samples are misclassified, the first two can be listed in the table.

EXPERIMENT 2: REINFORCEMENT LEARNING – Q-LEARNING

Objective

To implement the Q-Learning algorithm in a 4×4 Grid World environment and learn an optimal policy for reaching the goal while avoiding obstacles.

(a) Environment Definition

A **4×4 Grid World** is used for the experiment.

The environment contains a starting state, goal state and obstacle states.

Grid World

Position	Description
(0,0)	Starting state
(3,3)	Goal state
(1,1)	Obstacle
(2,2)	Obstacle

Grid Representation

	Column 0	Column 1	Column 2	Column 3
Row 0	Start	State	State	State
Row 1	State	Obstacle	State	State
Row 2	State	State	Obstacle	State

Row 3	State	State	State	Goal
--------------	-------	-------	-------	------

Available Actions

The agent can perform four actions.

Action	Symbol	Movement
Up	U	Row - 1
Down	D	Row + 1
Left	L	Column - 1
Right	R	Column + 1

Reward Structure

Situation	Reward
Reaching the goal	+10
Normal movement	-1
Moving into an obstacle	-5
Moving outside the grid	-1

The agent therefore has to learn a path that reaches the goal while reducing unnecessary movements and avoiding obstacles.

Q-Table Initialization

There are:

[
4 \times 4 = 16
]

states and 4 possible actions for each state.

Therefore, the Q-table contains:

[
16 \times 4 = 64
]

Q-values.

Initially, all Q-values are set to zero.

Hyperparameters

Hyperparameter	Symbol	Value
Learning Rate	α	0.1
Discount Factor	γ	0.9
Initial Exploration Rate	ϵ	1.0
Minimum Exploration Rate	$\epsilon_{\min}$	0.05
Episodes	—	100
Maximum steps per episode	—	100

Q-Learning Formula

The Q-value is updated using:

$$Q(s,a) = Q(s,a) + \alpha [r + \gamma \max_{a'} Q(s',a') - Q(s,a)]$$

where

Symbol	Meaning
(s)	Current state
(a)	Current action
(r)	Reward received
(s')	Next state
(\alpha)	Learning rate
(\gamma)	Discount factor

Python Code

```
import numpy as np

import matplotlib.pyplot as plt

# Grid size

grid_size = 4

# Actions: Up, Down, Left, Right

actions = [
```

```
(-1, 0),  
(1, 0),  
(0, -1),  
(0, 1)  
]
```

```
action_names = ["U", "D", "L", "R"]
```

```
# Start and goal
```

```
start = (0, 0)
```

```
goal = (3, 3)
```

```
# Obstacles
```

```
obstacles = {
```

```
(1, 1),
```

```
(2, 2)
```

```
}
```

```
# Q-table
```

```
Q = np.zeros((4, 4, 4))
```

```
# Hyperparameters
```

```
alpha = 0.1
```

```
gamma = 0.9
```

```
epsilon = 1.0
```

```
epsilon_min = 0.05
```

```
epsilon_decay = 0.98
```

```
np.random.seed(42)
```

```
def step(state, action):
```

```
    row, col = state
```

```
    dr, dc = actions[action]
```

```
    new_row = row + dr
```

```
    new_col = col + dc
```

```
    # Outside grid
```

```
    if not (0 <= new_row < 4 and 0 <= new_col < 4):
```

```
        return state, -1, False
```

```
    new_state = (new_row, new_col)
```

```
    # Obstacle
```

```
    if new_state in obstacles:
```

```
        return state, -5, False
```

```
    # Goal
```

```
    if new_state == goal:
```

```
        return new_state, 10, True
```

```

    # Normal movement

    return new_state, -1, False

rewards = []

q_values = {}

# Training

for episode in range(1, 101):

    state = start

    total_reward = 0

    for step_no in range(100):

        # Epsilon-greedy action selection

        if np.random.random() < epsilon:

            action = np.random.randint(4)

        else:

            action = np.argmax(

                Q[state[0], state[1]]

            )

        next_state, reward, done = step(

            state,

            action

        )

        old_value = Q[

            state[0],

```

```
        state[1],  
        action  
    ]  
  
    if done:  
        future_value = 0  
    else:  
        future_value = np.max(  
            Q[  
                next_state[0],  
                next_state[1]  
            ]  
        )
```

```
# Q-learning update
```

```
Q[  
    state[0],  
    state[1],  
    action  
] = old_value + alpha * (  
    reward +  
    gamma * future_value -  
    old_value  
)
```

```
total_reward += reward
```

```
state = next_state
```

```
if done:
```

```
    break
```

```
rewards.append(total_reward)
```

```
# Record Q-values
```

```
if episode in [1, 50, 100]:
```

```
    q_values[episode] = {
```

```
        "(0,0)": Q[0, 0].copy(),
```

```
        "(1,0)": Q[1, 0].copy()
```

```
    }
```

```
# Reduce exploration
```

```
epsilon = max(
```

```
    epsilon_min,
```

```
    epsilon * epsilon_decay
```

```
)
```

```
# Display Q-values
```

```
for episode, values in q_values.items():
```



```
print("\nEpisode:", episode)
```

```
print(  
    "(0,0):",  
    values["(0,0)"]  
)
```

```
print(  
    "(1,0):",  
    values["(1,0)"]  
)
```

```
# Display final policy
```

```
print("\nFinal Learned Policy:")
```

```
for r in range(4):
```

```
    row = []
```

```
    for c in range(4):
```

```
        if (r, c) == goal:
```

```
            row.append("G")
```

```
        elif (r, c) in obstacles:
```

```
            row.append("X")
```

```
    else:
```

```
        best_action = np.argmax(Q[r, c])
```

```
        row.append(  
            action_names[best_action]  
        )  
    print(row)  
  
# Plot cumulative reward  
cumulative_reward = np.cumsum(rewards)  
  
plt.plot(  
    range(1, 101),  
    cumulative_reward  
)  
plt.xlabel("Episode")  
plt.ylabel("Cumulative Reward")  
plt.title(  
    "Cumulative Reward per Episode"  
)  
plt.grid()  
plt.show()
```

IDLE Screenshot – Q-Learning Execution

```
Episode: 1
(0,0): [-0.1  0.  -0.1  -0.19]
(1,0): [0. 0.  0.  0.]

Episode: 50
(0,0): [-2.56636155 -2.27363778 -2.23239744 -2.09505365]
(1,0): [-2.03085261 -1.62657989 -1.8406771  -5.46631052]

Episode: 100
(0,0): [-2.17290758 -2.32520478 -1.9837654  1.14370864]
(1,0): [-1.96779997 -1.59417134 -1.90300158 -5.46631052]

Final Learned Policy:
['R', 'R', 'R', 'D']
['D', 'X', 'R', 'D']
['R', 'D', 'X', 'D']
['R', 'R', 'R', 'G']
```

Figure 4: Q-Learning training output

(b) Q-Value Evolution

The Q-values of two representative states are recorded at Episodes **1, 50 and 100**. The values are arranged as **[Up, Down, Left, Right]**.

State 1: (0,0)

Episode	Up	Down	Left	Right
1	-0.10	0.00	-0.10	-0.19
50	-2.5664	-2.2736	-2.2324	-2.0951
100	-2.1729	-2.3252	-1.9838	1.1437

State 2: (1,0)

Episode	Up	Down	Left	Right
1	0.00	0.00	0.00	0.00
50	-2.0309	-1.6266	-1.8407	-5.4663

100	-1.9678	-1.5942	-1.9030	-5.4663
-----	---------	---------	---------	---------

(c) Final Learned Policy

The final policy is obtained by selecting the action with the highest Q-value for each state.

Symbols

Symbol	Meaning
U	Up
D	Down
L	Left
R	Right
X	Obstacle
G	Goal

Cumulative Reward Graph

The cumulative reward is calculated using the rewards obtained during the 100 episodes.

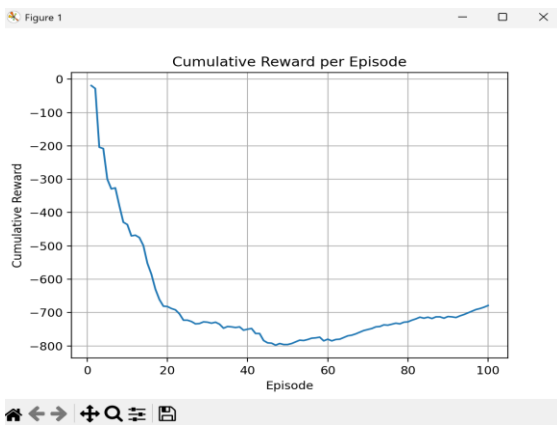


Figure 6: Cumulative reward per episode

Convergence Analysis

During the initial episodes, the agent explores different actions because the exploration rate is high. Therefore, the reward may vary significantly.

As the number of episodes increases, the agent receives more information about the environment. The Q-values are updated using the rewards obtained from previous actions.

The exploration rate is gradually reduced, allowing the agent to select the actions that have higher Q-values more frequently.

Therefore, the reward becomes comparatively more stable in the later episodes. This indicates that the agent has learned a useful policy for reaching the goal while avoiding the obstacle states.

The convergence may not be completely smooth because Q-Learning still performs some exploration during training.